

CS 423 — Computer Architecture

Bilkent University — Fall 2026

Homework 1: Single-Cycle vs In-Order Pipeline Architectures

100 points • Out: September 24, 2026 • Due: October 8, 2026, 23:59 (Ankara time)

Learning Objectives

After completing this homework, you will be able to:

- run `gem5` syscall-emulation (SE-mode) simulations in the pinned Docker environment and interpret `stats.txt` (instructions, cycles, CPI/IPC, simulated vs. host time);
- explain what `TimingSimpleCPU` (no overlap) and `MinorCPU` (4-stage in-order pipeline) each model, and where the “single-cycle” analogy for the former breaks down;
- quantify the throughput benefit of pipelining and reason about the clock-period side of the tradeoff using the iron law of performance;
- provoke, observe, and explain data hazards (especially load-use) and control hazards using microbenchmarks you write yourself;
- present measured results as clear plots with mechanism-level analysis.

GenAI Policy

GenAI tools are **allowed** as helpers for coding, preparing data visualization scripts, and proofreading your final text. They are **not allowed** to take over any task completely: implementing mechanisms, conducting simulations, analyzing data, or writing critical reviews. You are the sole author of everything you submit. In HW1 specifically: a tool may remind you of GCC inline-asm syntax, but the *hazard structure* of your Task 4 microbenchmarks and every prediction and explanation in your report must be your own. Your report must include a short GenAI-disclosure statement (see the report template).

Environment Setup

All experiments run in the course `gem5` Docker environment — `gem5 v25.1.0.1`, RISC-V, with the cross toolchain preinstalled (see `starter/README.md` for details and a build-it-yourself fallback):

```
docker pull s4lbot/cs423-gem5:v25.1.0.1
cd starter
docker run --rm -u "$(id -u):$(id -g)" -v "$PWD":/hw -w /hw \
  s4lbot/cs423-gem5:v25.1.0.1 bash scripts/smoke_test.sh
```

Results obtained with other `gem5` versions may differ and will not be regraded.

1 Background

Lectures L3b/L3c developed the in-order pipeline: overlapping instruction execution shortens the clock period from “slowest *instruction*” to “slowest *stage*”, at the price of hazards — data hazards (mitigated by forwarding, but not always), control hazards (mitigated by branch prediction, but not always), and structural hazards. In this homework you measure that tradeoff on a real simulator instead of taking the lecture’s word for it.

You will compare two gem5 CPU models on identical systems (same clock, same small L1 caches, same DDR3 memory):

Role	gem5 model	What it models
“single-cycle-like” baseline	TimingSimpleCPU	one instruction at a time, no overlap
in-order pipeline	MinorCPU	4-stage pipeline (Fetch1, Fetch2, Decode, Execute) with forwarding

An honest caveat, which is itself part of the assignment: TimingSimpleCPU is *not* a literal textbook single-cycle CPU. It executes instructions without overlap, but memory operations take realistic multi-cycle time, whereas a true single-cycle machine would stretch its clock period to cover the slowest instruction instead. We use it as the standard “no-overlap” baseline; Task 5 asks you to articulate exactly where this analogy holds and where it breaks.

2 Task 1: Environment bring-up and first simulation [10 points]

Set up the environment (setup box above) and run the smoke test. It cross-compiled the provided workloads and runs `hello` on the baseline CPU; results land in `results/smoke/stats.txt`.

Report the following from that run (**Q1.1–Q1.4**):

- Q1.1** `simInsts` — committed instructions;
- Q1.2** `system.cpu.numCycles` — CPU cycles simulated;
- Q1.3** `simSeconds` vs. `hostSeconds` — what does each measure, and why do they differ by orders of magnitude?
- Q1.4** Why is `simInsts` so much larger than the handful of instructions visible in `hello.c`? (Hint: static linking.)

3 Task 2: Characterize the non-pipelined baseline [20 points]

Using the *provided, complete* `configs/baseline.py` (TimingSimpleCPU, 1 GHz, 16 KiB L1I/L1D, DDR3-1600), run the three provided workloads and extract per-workload instructions, cycles, CPI, and simulated seconds (use `scripts/run_all.sh` and `scripts/extract_stats.py`; the pipeline runs in the script will fail until Task 3 — that is expected):

- **compute** — ALU-bound, independent operations;
- **memstream** — strided walk over 8 MiB, misses in L1;
- **branchy** — data-dependent, hard-to-predict branches.

Answer (**Q2.1–Q2.3**):

- Q2.1** CPI is well above 1 on every workload even though this CPU executes one instruction at a time with no hazards. Where do the extra cycles go? Use the per-workload differences as evidence.
- Q2.2** What would a *textbook single-cycle* machine do instead of spending multiple cycles on a load? What would that do to its clock period?
- Q2.3** Rank the three workloads by baseline CPI and explain the ranking from the workloads’ source code.

4 Task 3: Bring up the in-order pipeline and race it [25 points]

3.1 Complete `configs/pipeline.py` (8 pts)

The skeleton provides everything except the CPU: instantiate `RiscvMinorCPU`, attach the provided L1 caches, and create the interrupt controller (the `TODO(student)` block walks you through it;

`baseline.py` is your reference). Keep every non-CPU parameter identical to the baseline. Then run all three workloads on both CPUs (`scripts/run_all.sh`) and report CPI, IPC, and simulated-time speedup per workload (plots via `scripts/plot_results.py`).

3.2 The clock-period side of the bargain (7 pts)

Simulation compared the two CPUs *at the same clock*, which is unfair to the pipeline. Assume a classic 5-stage RISC datapath with component delays from Patterson & Hennessy¹:

Component	Delay
Instruction fetch (memory)	200 ps
Register read	100 ps
ALU operation	200 ps
Data access (memory)	200 ps
Register write	100 ps

- Q3.1** What clock period must a single-cycle implementation use, and what period can the pipelined implementation use? (Ignore pipeline-register overhead, or add a stated overhead — your choice, just be explicit.)
- Q3.2** Iron law: $\text{time} = \text{IC} \times \text{CPI} \times T_{\text{clk}}$. Using $\text{CPI} = 1$ for the single-cycle machine and your *measured* MinorCPU CPI for the pipeline, compute the end-to-end execution-time ratio for each workload. Which machine wins, where, and by how much?
- Q3.3** Your measured CPI was collected at 1 GHz with realistic memory latency. Explain why plugging it into Q3.2's re-clocked comparison is an approximation, and in which direction it biases the result for `memstream`.

3.3 Parameter probe (5 pts)

Vary *one* MinorCPU pipeline parameter via the provided `--minor-param` option of `pipeline.py`. Suggested choices — defaults and the full list are in the gem5 source tree, file `src/cpu/minor/BaseMinorCPU.py`:

- `executeInputWidth` (default 2) — recommended,
- `decodeInputWidth`, or `executeBranchDelay`.

Report CPI with and without your change on at least one workload and explain the mechanism behind the difference.

3.4 Find one stall in the pipeline trace (5 pts)

Capture a short `MinorTrace` window with `scripts/trace_pipeline.sh`, locate one cycle range where the pipeline stalls, quote those trace lines (a few lines, not pages) in your report, and explain what the pipeline was waiting for.

5 Task 4: Hazard microbenchmarks — provoke stalls on purpose [25 points]

This is the only from-scratch implementation task. Templates in `workloads/task4/` compile and run as shipped; you replace the placeholder kernels. For *each* experiment below, **write down your predicted outcome before running**, then measure, then explain — prediction, measurement, and mechanism carry the points; assembly elegance carries none.

¹D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed., Section 4.6 (the pipelining-overview example). The instructor may substitute the numbers used in lecture L3b — watch for an announcement; the method is what matters.

4.1 Load-use / dependency chain (13 pts)

In `loaduse.c`, implement two kernels with the *same* instruction count and mix (8 loads + 8 adds per iteration, in two groups of 4 + 4): a *dependent* kernel whose loads form a pointer chase — each load’s result is the address the next load dereferences (`ld p, 0(p)`; the scaffold pre-builds the pointer ring) — and an *independent* kernel loading the same array through fixed offsets. Everything stays in the L1 D-cache, so what you measure is pipeline behavior, not cache misses. Measure CPI for both kernels on *both* CPUs (4 runs; `run_all.sh` covers them). Explain: why does `TimingSimpleCPU` not care at all, while `MinorCPU` pays heavily for the chase? Why can forwarding not remove the wait for a load’s data, and — from your measured per-load extra cost — how much of the L1 hit latency does the in-order pipeline manage to hide in the independent version?

4.2 Predictable vs. unpredictable branches (12 pts)

In `brpair.c`, implement one loop body used with two precomputed condition arrays: a repeating pattern vs. pseudo-random bits. Measure on `MinorCPU`: CPI plus the two branch-predictor stats (`condPredicted` and `condIncorrect`, both under `system.cpu.branchPred` in `stats.txt`). Compute the misprediction rate for both variants and estimate the average penalty (in cycles) per misprediction from your CPI difference. What could the predictor learn, what could it not, and why? (`MinorCPU`’s default branch predictor is active out-of-the-box; treat it as a black box — HW2 opens it up.)

6 Task 5: Report [20 points]

One PDF following `report/report-template.md`: all plots (generated with the provided helper or your own), the answers to every question above, the “limits of the analogy” discussion (where `TimingSimpleCPU` misrepresents a true single-cycle machine and whether that changed your conclusions), and a reproducibility statement. Task 5 points reward the quality of interpretation and presentation; the raw numbers are already graded in Tasks 1–4.

Deliverables and Submission

Submit one archive `hw1_<studentID>.zip` via **Moodle** containing:

- **report.pdf** — Task 5, all analysis answers included;
- **configs/** — your completed `pipeline.py`;
- **workloads/** — your Task-4 sources; must build with `make -C workloads` in the course container;
- **results/** — one directory per reported run with its raw `stats.txt`, following the naming convention of `scripts/run_all.sh`: `<cpu>-<workload>[-<variant>]` (graders spot-check these against your report);
- **reproduce.sh** — regenerates every reported number and plot inside the pinned container with no manual steps.

Reproducibility is graded as a gate: if `reproduce.sh` fails to run in the pinned container, your total is capped at 50/100 (we will tell you and you may resubmit a fix within the late-policy window, with the late penalty applied).

Grading

Component	Points
Task 1: environment bring-up & stats scavenger hunt	10
Task 2: baseline characterization	20
Task 3: pipeline bring-up, iron law, probe, trace	25
Task 4: hazard microbenchmarks	25
Task 5: report quality & methodology	20
Total	100

Late policy: Late submissions are accepted with a **10% grade penalty per week** (applied per started week), up to the global late-submission cut-off for all labs: **December 14, 2026**. No submissions are accepted after the cut-off.

Collaboration policy: This homework is **individual** work. Partners are not allowed; all exercises must be completed and submitted individually. Discussing concepts is fine — sharing code, configurations, measurements, or report text is not.